

Batch: C3 Roll No.: 16010123325

Experiment No. 4

Grade: AA / AB / BB / BC / CC / CD / DD

Title: Implementing Platform as a Service - Creating a Web app and deploying it on Azure Cloud

Objective:

To Deploy a Python Web App on Microsoft Azure Platform and access the application on cloud through Browser

Expected Outcome of Experiment:

CO	Outcome
CO4	Build cloud services and applications

Books/ Journals/ Websites referred:

<http://surl.li/pkbbv>

<https://learn.microsoft.com/en-in/azure/app-service/quickstart-python?tabs=flask%2Cwindows%2Cazure-portal%2Cazure-cli-deploy%2Cdeploy-instructions-azportal%2Cterminal-bash%2Cdeploy-instructions-zip-azcli&pivots=python-framework-flask>

Related Theory: -

Infrastructure as a Service (IaaS) is a cloud computing service model in which virtualized computing resources such as servers, storage, networking, and operating systems are provided over the internet on a pay-as-you-go basis. The cloud service provider manages the physical infrastructure, while the consumer has control over:

- Virtual machines
- Operating systems
- Storage
- Applications
- Networking configuration (to a large extent)

Key Components of IaaS are as under-

- Compute – Virtual Machines (VMs)
- Storage – Block, Object, File storage
- Networking – Virtual networks, subnets, IP addresses
- Virtualization Layer – Hypervisor abstracts physical hardware
- Management Interface – Web portal, CLI, APIs

Responsibilities: Provider vs Consumer

Layer	Managed By
Physical Data Center	Cloud Provider
Servers & Storage Hardware	Cloud Provider
Virtualization (Hypervisor)	Cloud Provider
Operating System	Consumer
Middleware	Consumer
Runtime & Applications	Consumer
Data	Consumer

Characteristics of Infrastructure as a Service (IaaS)

1. On-demand self-service

Users can provision computing resources such as virtual machines and storage automatically without human interaction with the cloud provider. Resources can be created, modified, or deleted anytime using a web portal or API.

2. Scalability & Elasticity

IaaS allows resources to be scaled up or down based on workload requirements. This ensures optimal performance during peak demand and cost savings during low usage.

3. Resource Pooling

The cloud provider pools physical resources and dynamically allocates them to multiple users using virtualization. Customers share the same infrastructure while remaining logically isolated from each other.

4. Measured Service

Resource usage is continuously monitored, measured, and billed based on actual consumption. Users pay only for the compute, storage, and network resources they use.

5. High Availability

IaaS platforms provide reliable access to resources through redundant data centers and fault-tolerant architecture. This ensures minimal downtime and continuous service availability.

Implementation Details:

1. Follow steps on available on this link:
<http://surl.li/pkbbv>
2. Create a sample application (a web app) on a local machine using git cloning.
3. Create a web service on Azure for hosting web app made in step 1. (USE AZURE PORTAL, NOT AZURE CLI)
4. Deploy code for application made in Step 1 on Azure cloud (DEPLOY USING LOCAL GIT)
5. Access app deployed on cloud through Browser

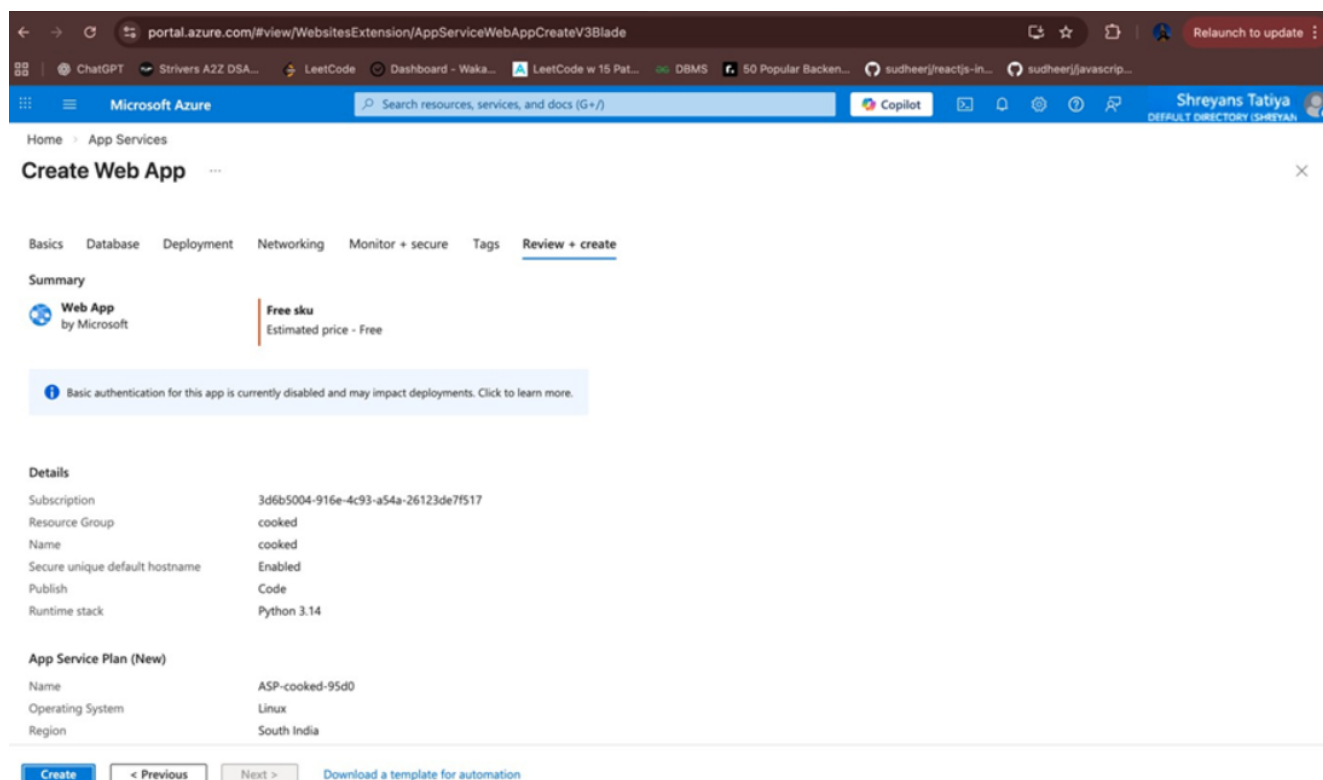
A. Include step wise screenshots in the document with proper heading for each screenshot.

B. For important screenshots, also explain what is happening in the step

Implementation

Step 1: Configure Basic Details in Azure Portal

Screenshot 1: Create Web App – Basics Page



Explanation:

In this step, the basic configuration for the Web App is defined.

The “Secure unique default hostname” option ensures that Azure generates a secure HTTPS-enabled URL for the web app.

Step 2: Configure Pricing and Region

Screenshot 2: Region and Pricing Plan Selection

Home > App Services

Create Web App

Basics Database Deployment Networking Monitor + secure Tags Review + create

App Service Web Apps lets you quickly build, deploy, and scale enterprise-grade web, mobile, and API apps running on any platform. Meet rigorous performance, scalability, security and compliance requirements while using a fully managed platform to perform infrastructure maintenance. [Learn more](#)

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * Azure for Students

Resource Group * (New) cooked [Create new](#)

Instance Details

Name cooked
 -h9bagxbztd8buen.southindia-01.azurewebsites.net

☒ Secure unique default hostname on. [More about this update](#)

Publish * ☒ Code ☐ Container

Runtime stack * Python 3.14

Operating System * ☒ Linux ☐ Windows

[Review + create](#) < Previous Next : Database >

Explanation:

In this step, hosting location and pricing plan are selected.

Step 3: Review and Create Web App

Screenshot 3: Review + Create Page

Explanation:

This page displays a summary of all configurations including:

- Subscription
- Resource Group
- Runtime Stack (Python 3.14)
- OS (Linux)
- App Service Plan
- Region

Azure validates the configuration.

Home / App Services

Create Web App

Operating System * Linux Windows

Region * South India

Not finding your App Service Plan? Try a different region or select your App Service Environment.

Pricing plans
 App Service plan pricing tier determines the location, features, cost and compute resources associated with your app. [Learn more](#)

Linux Plan (South India) * (New) ASP-cooked-95d0 [Create new](#)

Pricing plan Free F1 (Shared infrastructure) [Explore pricing plans](#)

Zone redundancy
 An App Service plan can be deployed as a zone redundant service in the regions that support it. Your initial instance count will be set based on your zone redundancy configuration. To ensure you'll be able to enable zone redundancy at any point in the lifecycle of your app, enable zone redundancy now. You can decrease the instance count after the App Service plan is created. [Learn more](#)

Zone redundancy

☐ Enabled: Your App Service plan and the apps in it will be zone redundant. The minimum App Service plan instance count will be two.

☒ Disabled: Your App Service plan and the apps in it will not be zone redundant. The minimum App Service plan instance count will be one.

[Review + create](#) [< Previous](#) [Next : Database >](#)

Step 4: Configure Deployment Center

Screenshot 4: Azure Deployment Center Settings

Home / cooked

cooked | Deployment Center

Web App

Search

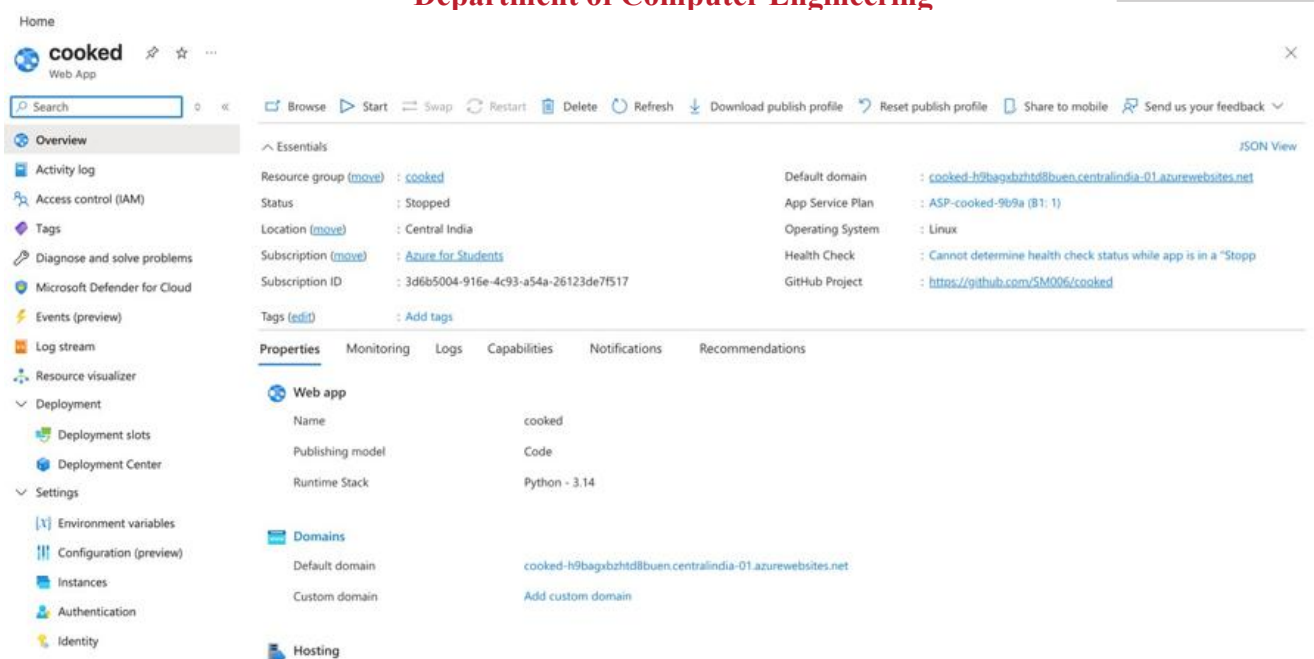
Overview Activity log Access control (IAM) Tags Diagnose and solve problems Microsoft Defender for Cloud Events (preview) Log stream Resource visualizer Deployment Deployment slots **Deployment Center** Settings [x] Environment variables Configuration (preview) Instances Authentication Identity

Settings Containers (new) Logs FTPS Credentials

Save Discard Refresh Browse Sync Send us your feedback

Deploy and build code from your preferred source and build provider. [Learn more](#)

Source	GitHub
Organization	SM006
Repository	cooked
Branch	main
Build	
Build provider	GitHub Actions
Runtime stack	Python
Version	3.14



Explanation:

In this step, the Web App is connected to a GitHub repository for automatic deployment.

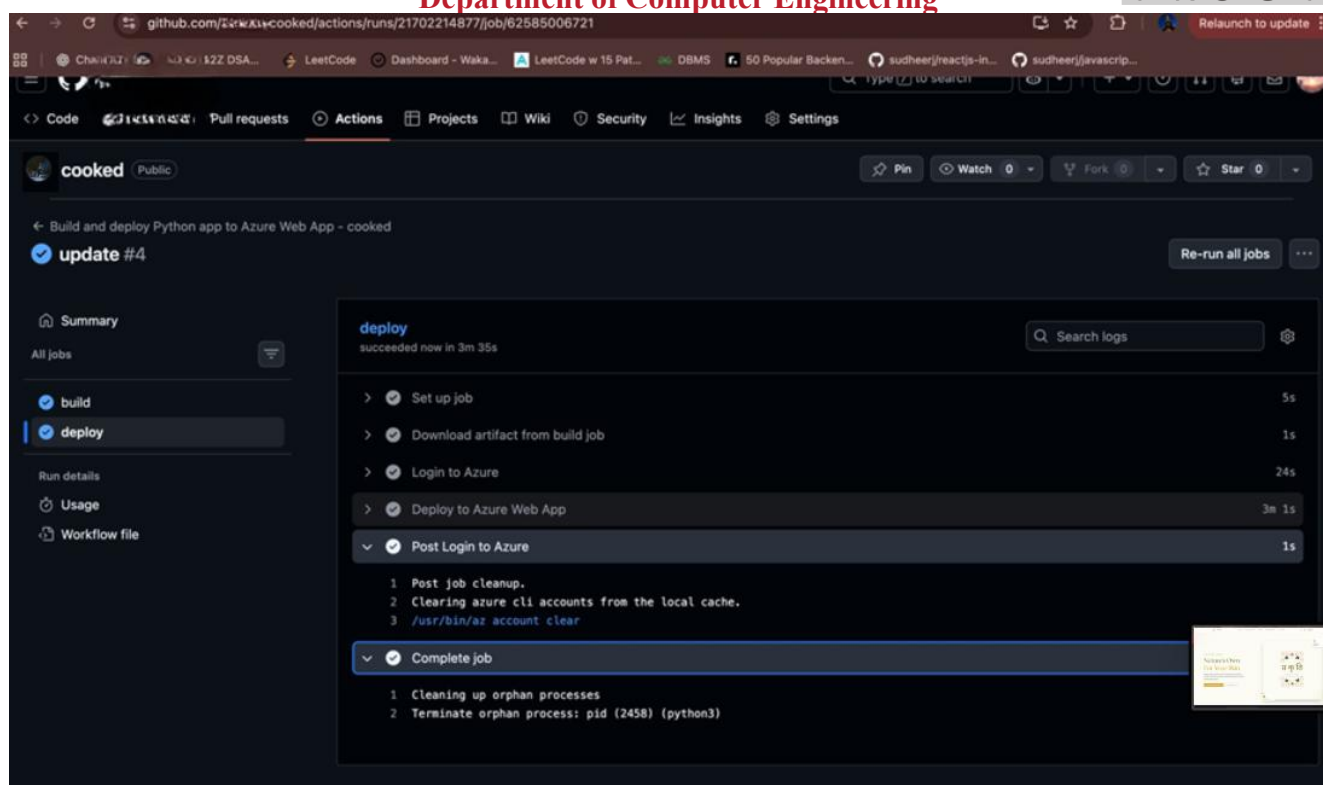
Configuration includes:

- **Source:** GitHub
- **Organization:** SM006
- **Repository:** cooked
- **Branch:** main
- **Build Provider:** GitHub Actions
- **Runtime Stack:** Python 3.14

This enables Continuous Integration and Continuous Deployment (CI/CD). Whenever code is pushed to the main branch, GitHub Actions automatically builds and deploys the application to Azure.

Step 5: GitHub Actions Deployment Workflow

Screenshot 5: GitHub Actions – Build and Deploy Workflow



Explanation:

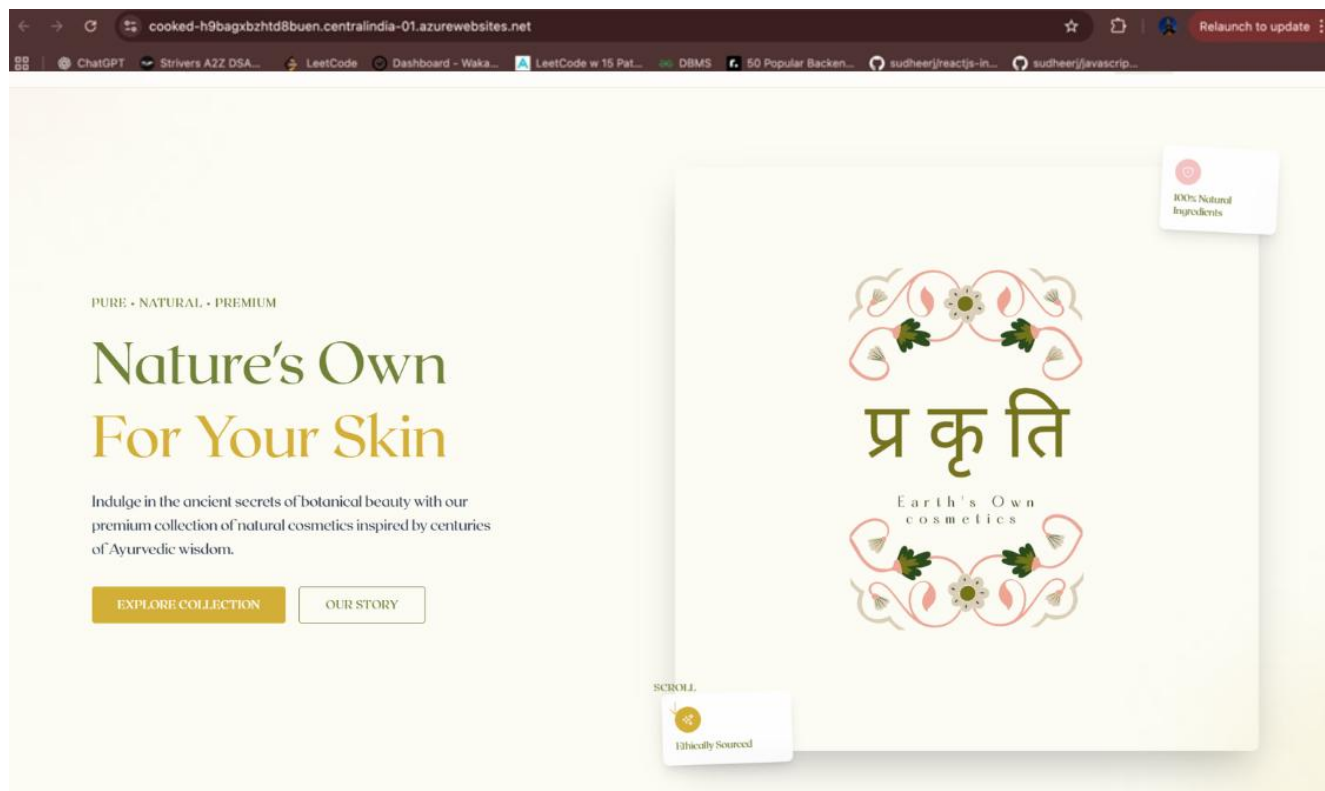
The GitHub Actions workflow performs the following steps:

1. Set up job environment
2. Download build artifacts
3. Login to Azure
4. Deploy to Azure Web App
5. Post login cleanup

The deployment status shows **Succeeded**, indicating the web application has been successfully deployed.

Step 6: Deployment Success Confirmation

Screenshot 6: Deployment Completed Successfully



Explanation:

This confirms that the Python web application is now successfully hosted on Microsoft Azure and accessible via the generated web URL.

Post Lab Questions:

1. **How does deploying a web application on Azure IaaS differ from deploying it on a Platform as a Service (PaaS) model? (Compare responsibility, control, and management overhead.)**

In **IaaS**, the user is responsible for managing the virtual machine, operating system, installed software, and security configurations. Azure only provides the underlying infrastructure such as servers, storage, and networking. It offers complete control but increases management overhead.

In **PaaS**, Azure manages the operating system, runtime environment, scaling, and updates. The user only focuses on deploying and managing the application code. This reduces operational effort but limits low-level system control.

Comparison:

- **Responsibility:**
IaaS – User manages OS and environment
PaaS – Azure manages OS and infrastructure
- **Control:**
IaaS – Full system-level control
PaaS – Limited configuration control
- **Management Overhead:**
IaaS – High
PaaS – Low

2. **Why is Infrastructure as a Service (IaaS) suitable for deploying web applications that require full control over the operating system? (Consider OS selection, software installation, and configuration control.)**

IaaS is suitable when applications require full control over the operating system. It allows users to select the OS version, install custom software, configure system settings, manage security policies, and modify server-level configurations. This makes IaaS ideal for applications that depend on specific software versions, custom configurations, or advanced system-level control.

Conclusion:-

Through this experiment, we learn how to deploy a Python web application on Microsoft Azure using Azure App Service. The experiment helps understand the difference between IaaS and PaaS models and how cloud platforms simplify application hosting through managed services.